

Tuần 6 — Security I — IAM + STS + Cognito

Domain: Domain 2 – Security (26%) · **Thời lượng:** ~11h (4 buổi) · **Vị trí:** Tuần 1/2 của Domain 2

Điều hướng:  Tuần 5 ·  Kế hoạch tổng · Tuần 7 

Mục tiêu tuần này

- **Đọc được** một **IAM** policy JSON và nói ngay ai được/không được làm gì, kể cả khi có cả **Allow** lẫn **Deny**.
- **Giải thích được** quy tắc đánh giá quyền: explicit **Deny** > explicit **Allow** > implicit **Deny**.
- **Phân biệt được** identity-based policy vs resource-based policy (có **Principal**) và biết khi nào dùng cái nào.
- **Tự tay** viết IAM policy least-privilege cho **Lambda** và kiểm chứng một **Deny** chặn được **Action** dù vẫn có **Allow**.
- **Chọn đúng** API **STS** cho từng tình huống (cross-account, federation OIDC/SAML, MFA).
- **Phản xạ trong 5 giây:** "đăng nhập app / trả JWT" → **Cognito User Pool**; "client gọi thẳng service AWS / cần AWS credentials" → **Cognito Identity Pool**.

Nội dung học chi tiết

Buổi A — Lý thuyết (~3h)

1. Cấu trúc một **IAM** policy (rất hay hỏi cú pháp)

- Policy là JSON gồm **Version** (dùng **"2012-10-17"**) và một hoặc nhiều **Statement**.
- Mỗi **Statement** có các phần tử:

Phần tử	Ý nghĩa
Effect	Allow hoặc Deny
Action	thao tác trên service, vd <code>s3:GetObject</code> , <code>dynamodb:PutItem</code> (hỗ trợ wildcard <code>*</code>)
Resource	ARN của tài nguyên áp dụng
Condition	điều kiện có thì statement mới hiệu lực (vd <code>aws:SourceArn</code> , <code>aws:MultiFactorAuthPresent</code> , <code>StringEquals...</code>)
Principal	ai được áp dụng — chỉ có trong resource-based policy

2. Quy tắc đánh giá quyền (câu hỏi kinh điển)

- Thứ tự ưu tiên: **explicit Deny** > **explicit Allow** > **implicit Deny**.
- Chỉ cần MỘT explicit Deny** là request bị chặn — **bất kể** có bao nhiêu **Allow** khác cho cùng **Action**.
- Không khai báo gì → mặc định là **implicit Deny** (mọi thứ đều bị từ chối cho tới khi được **Allow** tường minh).
- ⇒ Kết luận nhanh: policy vừa có **Allow** vừa có **Deny** cho cùng một **Action** → **kết quả là Deny**.

3. Identity-based vs Resource-based policy

	Identity-based	Resource-based
Gắn vào	user / group / role	chính tài nguyên (bucket, function, queue, topic...)
Có Principal ?	KHÔNG (đã biết chủ thể là identity gắn vào)	CÓ (phải chỉ rõ AI được phép)
Ví dụ	policy gắn vào role của Lambda	S3 bucket policy, Lambda resource policy, SQS/SNS access policy
Dùng cho	cấp quyền cho chủ thể trong account	cấp quyền cross-account / cho principal bên ngoài truy cập tài nguyên

- Ghi nhớ: thấy phần **"Principal"** trong JSON → chắc chắn là **resource-based policy**.

4. Roles gắn với compute (dễ nhầm)

- **Lambda execution role:** quyền để **function gọi service khác** (đọc **DynamoDB**, ghi **S3**, publish **SNS**...). Function tự nhận temporary credentials từ role này, không nhúng access key vào code.
- **EC2 instance profile:** "vỏ bọc" để gắn một **IAM** role vào EC2 instance; ứng dụng trên EC2 lấy credentials tạm qua instance metadata.
- **ECS task role vs task execution role** (rất hay bẫy):

Role	Cấp quyền cho	Dùng để
Task role	ứng dụng bên trong container	code trong container gọi service AWS (vd đọc S3 , ghi DynamoDB)
Task execution role	ECS agent (hạ tầng)	kéo image từ ECR + ghi log lên CloudWatch Logs , đọc secret khi khởi động task

B Buổi B — Hands-on (~3.5h)

 Lab cầm tay chỉ việc (từng bước + lệnh + code): labs.md.

Lab 1 — Viết IAM policy least-privilege cho **Lambda** + test explicit **Deny**

1. Tạo policy identity-based chỉ cho phép ghi vào **MỘT** bảng **DynamoDB** cụ thể:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowPutOrders",
      "Effect": "Allow",
      "Action": ["dynamodb:PutItem", "dynamodb:GetItem"],
      "Resource": "arn:aws:dynamodb:ap-southeast-1:111122223333:table/Orders"
    }
  ]
}
```

2. Gắn policy vào **Lambda** execution role. Test function ghi được vào bảng **Orders**, nhưng **không** ghi được vào bảng khác (implicit Deny).
3. **Chứng minh explicit Deny thắng:** thêm statement **Deny** cho chính **dynamodb:PutItem** (có thể kèm **Condition**):

```
{
  "Sid": "BlockDeletes",
  "Effect": "Deny",
  "Action": "dynamodb:DeleteItem",
  "Resource": "*"
}
```

Dù có **Allow** rộng ở đâu đó, **DeleteItem** vẫn bị chặn tuyệt đối.

4. Dùng **IAM Policy Simulator** (console) để mô phỏng: nhập **Action** + **Resource**, xem kết quả **allowed/denied** và statement nào quyết định.

Lab 2 — **aws sts assume-role** (cross-account hoặc mô phỏng)

1. Ở account B (account đích), tạo role có **trust policy** cho phép principal của account A (**Principal** = account A) **sts:AssumeRole**.
2. Gắn cho role đó quyền cần thiết (vd đọc một bucket **S3** của account B).
3. Ở account A, gọi:

```
aws sts assume-role \
  --role-arn arn:aws:iam::<ACCOUNT_B>:role/CrossAccountRead \
  --role-session-name demo-session
```

4. Lệnh trả về **temporary credentials**: **AccessKeyId** + **SecretAccessKey** + **SessionToken**. Export cả 3 vào biến môi trường rồi gọi thử service của account B.
5. Xác nhận danh tính hiện tại:

```
aws sts get-caller-identity
```

Lab 3 — **Cognito User Pool** + **Hosted UI** → nhận JWT

1. Tạo một **User Pool**, bật **MFA** (tùy chọn) và tạo một App Client.
2. Bật **Hosted UI**, cấu hình domain + callback URL, chọn OAuth scopes.
3. Đăng nhập qua Hosted UI → sau khi xác thực, **Cognito** trả về **3 JWT**: **ID token**, **Access token**, **Refresh token**.
4. Giải mã ID/Access token (jwt.io) để xem claims. Ghi nhớ: **ID/Access token mặc định hết hạn 1 giờ**; refresh token dùng để lấy token mới, thời hạn **cấu hình được**.

Lab 4 — **Cognito Identity Pool** → nhận temporary AWS credentials

- 1. Tạo một **Identity Pool**, gắn **User Pool** ở Lab 3 làm authentication provider.
- 2. Cấu hình 2 role: **authenticated** và **unauthenticated** (guest), mỗi role có IAM policy least-privilege.
- 3. App đổi JWT (từ **User Pool**) lấy **temporary AWS credentials** qua **STS**.
- 4. Dùng credentials đó gọi TRỰC TIẾP một service AWS (vd đọc một bảng **DynamoDB**) — đây chính là kịch bản "client gọi thẳng AWS".

C Buổi C — Bổ sung (~2.5h)

1. **STS** — chọn đúng API cho từng tình huống

API STS	Dùng khi	Trả về
AssumeRole	đổi vai, thường cross-account hoặc same-account	temporary credentials (access key + secret + session token)
AssumeRoleWithWebIdentity	liên kết OIDC (Cognito / Google / Facebook...)	temporary credentials
AssumeRoleWithSAML	SAML IdP doanh nghiệp (AD FS, Okta SAML...)	temporary credentials
GetSessionToken	phiên tạm cho chính user, thường kèm MFA	temporary credentials

- Điểm chung: mọi API **STS** đều trả **credentials TẠM THỜI** (có **SessionToken**), không phải key vĩnh viễn.
- **Regional endpoints (sắc thái mới):** AWS CLI v2 & SDK mới **mặc định dùng regional STS endpoint** (khuyến nghị: giảm latency, resilient hơn global). Region **enabled-by-default** tự động active **STS**; lỗi **RegionDisabled** chủ yếu xảy ra với **opt-in region**.

2. **Cognito** — hai thành phần khác vai trò hoàn toàn

	User Pool	Identity Pool (Federated Identities)
Vai trò	Authentication — xác thực người dùng	Authorization — cấp quyền AWS
Trả về	JWT (ID / Access / Refresh token)	temporary AWS credentials (qua STS)
Dùng để	đăng nhập app, quản lý user directory	client gọi trực tiếp service AWS
Hỗ trợ	social / SAML / OIDC federation, MFA, Hosted UI, groups, Lambda triggers	map identity → IAM role (authenticated & unauthenticated)

- **User Pool chi tiết:** là user directory + xác thực; trả 3 JWT; **ID/Access token mặc định 1 giờ**, refresh token cấu hình được. Hỗ trợ **federation** (SAML/OIDC/social), **MFA**, **Hosted UI**, **groups** (gom user để phân quyền), và **Lambda triggers** (Pre Sign-up, Post Confirmation, Pre Token Generation... để tùy biến luồng auth).
- **Hosted UI vs Managed Login:** **Hosted UI** = giao diện classic (first-gen). **Managed Login** (ra mắt 2024) = thế hệ mới: **branding editor no-code**, hỗ trợ **passkey**, **dark mode**; cần feature plan **Essentials/Plus** (Lite chỉ có **Hosted UI** classic). **Plus** thêm **threat protection / adaptive auth**.
- **Identity Pool chi tiết:** nhận token (từ **User Pool** hoặc IdP ngoài) → đổi lấy **temporary AWS credentials** qua **STS**, map vào IAM role. Có role riêng cho user đã đăng nhập và khách (**guest/unauthenticated**).

3. Luồng kết hợp điển hình (đề rất thích)

- App mobile: đăng nhập qua **User Pool** (nhận JWT) → đưa JWT cho **Identity Pool** → nhận **AWS credentials tạm** → gọi thẳng **DynamoDB/S3**. **User Pool** lo ai bạn là, **Identity Pool** lo bạn được làm gì trên AWS.

Đọc thêm: **IAM** policy evaluation logic, **IAM** JSON policy reference, **STS** API reference, **Cognito** Developer Guide (User Pool vs Identity Pool).

Buổi D — Practice + Review (~2h)

 **Bộ câu hỏi luyện tập của tuần:** [questions.md](#) — đáp án & giải thích: [answers.md](#). (bằng tiếng Anh — văn phong đề thật để làm quen đề.)

- Làm bộ câu hỏi chủ đề **IAM** policy evaluation, roles (Lambda/ECS), **STS**, **Cognito**.
- **Ghi sổ câu sai**, phân loại theo: đọc policy / chọn role / chọn STS API / User Pool vs Identity Pool.
- **Spaced repetition**: ôn flashcard theo mốc **1 / 3 / 7 ngày** — nhất là bảng STS API và cặp task role vs execution role (rất dễ quên).



PHẢI NHỚ tuần này

Fact	Ghi nhớ
Thứ tự đánh giá quyền	explicit** Deny ** > explicit Allow > implicit Deny
Cùng action có cả Allow + Deny	KẾT QUẢ = Deny (một Deny là chặn, bất kể bao nhiêu Allow)
Không khai báo	implicit Deny (mặc định từ chối)
Principal	chỉ có trong resource-based policy (chỉ rõ AI được phép)
Resource-based ví dụ	S3 bucket policy, Lambda resource policy, SQS/SNS access policy
Lambda execution role	quyền để function gọi service khác
EC task role	quyền cho ứng dụng bên trong container
EC task execution role	quyền cho ECS agent kéo image từ ECR + ghi CloudWatch Logs
AssumeRole	đổi vai (cross/same-account) → temp creds (key + secret + session token)
AssumeRoleWithWebIdentity	federation OIDC (Cognito/Google/Facebook)
AssumeRoleWithSAML	SAML IdP doanh nghiệp
GetSessionToken	phiên tạm, thường kèm MFA
Cognito User Pool	authentication → 3 JWT (ID/Access/Refresh); ID/Access mặc định 1 giờ
Hosted UI vs Managed Login	Hosted UI = classic (first-gen); Managed Login (2024) = no-code branding + passkey +

Fact	Ghi nhớ
	dark mode , cần plan Essentials/Plus (Lite chỉ có Hosted UI); Plus thêm threat protection
Cognito Identity Pool	authorization → temp AWS creds qua STS , map IAM role (authenticated & unauthenticated)
Phản xạ	"trả JWT/đăng nhập" → User Pool ; "gọi thẳng AWS/cần creds" → Identity Pool

⚠ Bẫy đề hay gặp

- Thấy "policy có cả **Allow** và **Deny** cho cùng action" → tưởng lấy **Allow** → **SAI**, explicit **Deny** luôn thắng → kết quả **Deny**.
- Thấy "user không được gán policy nào cho action X" → tưởng được ngầm cho phép → **SAI**, mặc định là **implicit Deny**.
- Thấy "**Principal**" trong đề → tưởng identity-based → **SAI**, có **Principal** là **resource-based policy**.
- Cần cấp quyền **cross-account** cho tài nguyên → tưởng phải tạo user ở account kia → dùng **resource-based policy** (hoặc **AssumeRole** cross-account) chuẩn hơn.
- Container không kéo được image từ **ECR** / không ghi được log → sửa **task role** là **SAI** → phải sửa **task execution role**.
- App trong container không gọi được **DynamoDB** → sửa execution role là **SAI** → phải sửa **task role**.
- Mobile app cần gọi thẳng **DynamoDB** → chọn **User Pool** là **SAI** → phải là **Identity Pool** (đổi token lấy AWS credentials).
- Federation với Google/Facebook (OIDC) → chọn **AssumeRoleWithSAML** là **SAI** → dùng **AssumeRoleWithWebIdentity**.
- Nhúng access key vĩnh viễn vào **Lambda/EC2** → **SAI** → dùng **role** (execution role / instance profile) để nhận credentials tạm.

🔄 Phản xạ nhanh (keyword → đáp án)

Thấy từ khoá	Bật ngay
cùng action vừa Allow vừa Deny	Deny (explicit Deny thắng)

Thấy từ khoá	Bật ngay
không được cấp gì	implicit Deny
policy có Principal	resource-based policy
cấp quyền chéo account cho một tài nguyên	resource-based policy / AssumeRole cross-account
function Lambda cần gọi service khác	Lambda execution role
container kéo image ECR / ghi CloudWatch Logs	ECS task execution role
app bên trong container gọi service AWS	ECS task role
đổi vai / lấy credentials tạm (cross-account)	STS AssumeRole
federation OIDC (Cognito/Google/Facebook)	AssumeRoleWithWebIdentity
federation SAML doanh nghiệp	AssumeRoleWithSAML
phiên tạm kèm MFA	GetSessionToken
đăng nhập app / cần JWT / user directory	Cognito User Pool
client gọi thẳng AWS service / cần AWS credentials	Cognito Identity Pool
guest (chưa đăng nhập) vẫn cần quyền AWS hạn chế	Identity Pool — unauthenticated role



Lab checklist

- ☐ Viết IAM policy least-privilege cho **Lambda**, xác nhận chỉ làm được đúng action cho phép.
- ☐ Thêm explicit **Deny** và chứng minh nó chặn action dù vẫn còn **Allow**.
- ☐ Dùng IAM Policy Simulator kiểm tra một request **allowed/denied**.
- ☐ **aws sts assume-role** (cross-account hoặc mô phỏng) → nhận và dùng temporary credentials.
- ☐ Dựng **Cognito User Pool** + Hosted UI, đăng nhập và lấy về 3 JWT.

- □ Dùng **Cognito Identity Pool**, đổi JWT lấy temporary AWS credentials và gọi một service AWS.

Cổng tự kiểm tra (phải trả lời trôi chảy mới sang tuần sau)

- **Mobile app cần gọi TRỰC TIẾP DynamoDB → dùng User Pool hay Identity Pool? Đáp án gọn: Identity Pool** — nó đổi token lấy temporary AWS credentials (qua STS) để client gọi thẳng AWS. User Pool chỉ xác thực và trả JWT. (Thực tế đăng nhập bằng User Pool rồi đưa JWT cho Identity Pool.)
- **Khi policy vừa có Allow vừa có Deny cho cùng một action → kết quả? Đáp án gọn: Deny.** Explicit Deny luôn thắng; chỉ cần một Deny là chặn, bất kể bao nhiêu Allow.
- **Cross-account access dùng STS API nào? Đáp án gọn: AssumeRole** (account đích tạo role có trust policy cho account nguồn) → trả temporary credentials (key + secret + session token).
- **ECS task role vs task execution role khác nhau chỗ nào? Đáp án gọn:** task role = quyền cho ứng dụng trong container gọi service AWS; task execution role = quyền cho ECS agent kéo image từ ECR + ghi CloudWatch Logs.
- **Làm sao biết một policy là resource-based hay identity-based? Đáp án gọn:** có phần Principal → resource-based (chỉ rõ AI được phép); không có Principal → identity-based.
- **Cognito User Pool trả về gì và hết hạn bao lâu? Đáp án gọn:** 3 JWT — ID / Access / Refresh token; ID/Access token mặc định hết hạn 1 giờ, refresh token cấu hình được.

Tài nguyên tuần này

 **Đã crawl sẵn tài liệu AWS vào resources/** — đọc offline được.

- AWS Docs: **AWS IAM** User Guide — Policy evaluation logic, JSON policy elements (Effect/Action/Resource/Condition/Principal).
- AWS Docs: **AWS IAM** User Guide — Identity-based vs resource-based policies; roles cho Lambda, EC2 instance profile, ECS task/execution roles.
- AWS Docs: **AWS STS** API Reference — AssumeRole, AssumeRoleWithWebIdentity, AssumeRoleWithSAML, GetSessionToken;

cross-account access.

- AWS Docs: [Amazon Cognito](#) Developer Guide — User Pools (JWT, MFA, Hosted UI, groups, Lambda triggers) vs Identity Pools (temporary AWS credentials, authenticated/unauthenticated roles).
- FAQ: [AWS IAM](#) FAQs, [Amazon Cognito](#) FAQs.
- Khoá học: Stephane Maarek — mục [IAM](#) advanced + [STS](#) + [Cognito](#); Adrian Cantrill — Identity & federation, Cognito deep-dive.



Checklist hoàn thành Tuần 6

- ☐ Hoàn thành 4 buổi A/B/C/D
- ☐ Thuộc lòng quy tắc đánh giá quyền (Deny > Allow > implicit Deny) và bảng STS API
- ☐ Phân biệt được identity-based vs resource-based, task role vs execution role
- ☐ Phân biệt được [Cognito User Pool](#) vs [Identity Pool](#) qua keyword
- ☐ Hoàn thành 6 lab (IAM policy + Deny, Policy Simulator, [assume-role](#), User Pool + Hosted UI, Identity Pool)
- ☐ Vượt Cổng tự kiểm tra